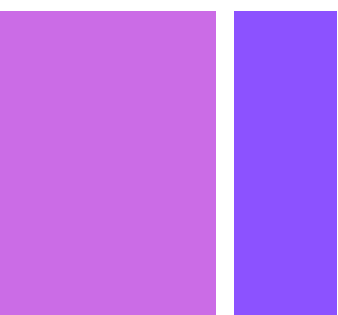




CONFIG_EXT4_FS

O kernel não entende "discos" — ele entende bytes. O filesystem é a camada que diz como esses bytes estão organizados em arquivos e diretórios.

Sem isso:

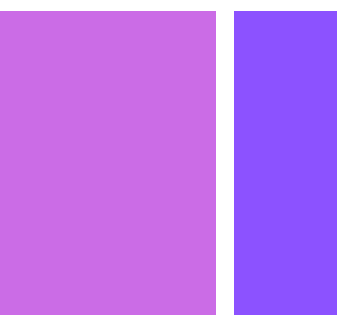
kernel encontra o disco → lê bytes brutos → não sabe o que é arquivo

→ não consegue montar o rootfs → kernel panic

Se o seu disco estiver formatado em ext4, precisa de CONFIG_EXT4_FS=y. Se for btrfs, CONFIG_BTRFS_FS=y.

O kernel precisa entender exatamente o formato do disco onde está o rootfs.





BLOCK DEVICE — CONFIG_BLK_DEV

É a camada de abstração entre o kernel e os dispositivos de armazenamento. Ela expõe os discos como arquivos em /dev/sda, /dev/nvme0n1, etc.

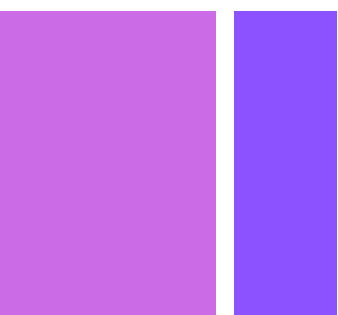
Sem isso:

kernel → não enxerga nenhum disco como dispositivo

→ /dev/sda não existe → não tem o que montar

É o subsistema que transforma "controlador de disco" em "arquivo que posso ler e escrever".





SCSI — CONFIG_SCSI

SCSI (Small Computer System Interface) é um protocolo antigo que virou o padrão de comunicação entre o kernel e dispositivos de armazenamento. Discos SATA modernos ainda usam SCSI por baixo — o AHCI traduz os comandos.

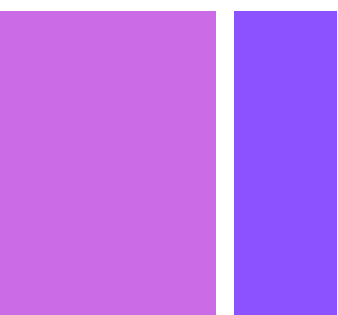
Sem isso:

AHCI manda comandos SCSI para o disco

- kernel não tem o subsistema SCSI
- não consegue falar com o controlador
- disco invisível

É o intermediário entre o block device e o controlador físico.





SATA — CONFIG_SATA_AHCI

AHCI = Advanced Host Controller Interface. É o driver do controlador físico SATA — o chip na placa-mãe que coordena a transferência de dados entre CPU, RAM e disco.

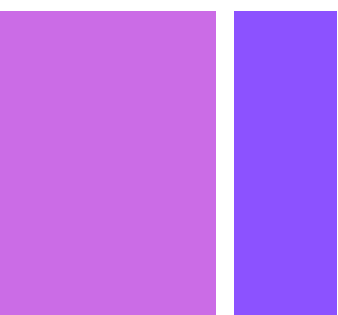
Sem isso:

placa-mãe tem controlador SATA

- kernel não tem driver para ele
- não consegue enviar/receber dados do disco
- disco invisível mesmo com SCSI e BLK_DEV habilitados

A cadeia é: BLK_DEV → SCSI → AHCI → disco físico. Faltando qualquer elo, o disco não aparece.





NVME — CONFIG_BLK_DEV_NVME

Discos NVMe (SSDs modernos M.2 e PCIe) não usam SATA. Eles se comunicam diretamente pelo barramento PCIe com o protocolo NVMe. O driver AHCI não tem efeito nenhum sobre eles.

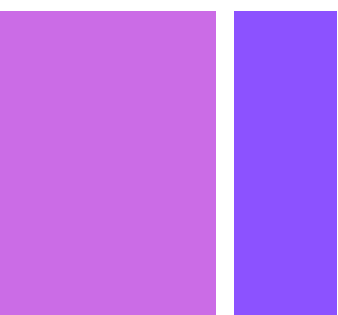
Sem isso:

sistema tem SSD NVMe

- kernel só tem driver AHCI (SATA)
- NVMe usa protocolo completamente diferente
- disco invisível

Se seu computador tem SSD NVMe, CONFIG_SATA_AHCI sozinho não resolve nada. Você precisa de CONFIG_BLK_DEV_NVME.





PROCFS — CONFIG_PROC_FS

Cria o pseudo-filesystem /proc. Não existe em disco — é gerado em runtime pelo kernel.
Sem isso:

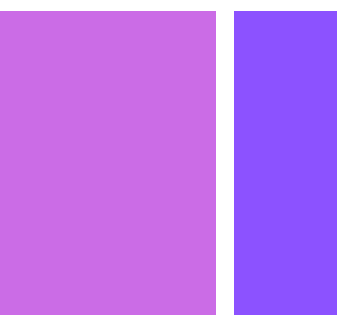
sem /proc/cpuinfo → ferramentas não sabem quantos cores existem

sem /proc/meminfo → não sabe quanta RAM há

sem /proc/[pid]/ → processos não conseguem informação uns dos outros

Muitos programas leem /proc diretamente. O htop, ps, top, free todos dependem de /proc. O próprio BusyBox usa /proc para vários applets.





SYSFS — CONFIG_SYSFS

Cria o pseudo-filesystem /sys. Expõe a estrutura interna de hardware e drivers do kernel.
Sem isso:

sem /sys/bus/ → udev/mdev não consegue detectar hardware

sem /sys/class/ → sem informação de dispositivos de bloco, rede, etc.

→ hotplug não funciona (plugar pendrive não faz nada)

É o que permite o sistema saber quando você conecta ou desconecta hardware em runtime.



DEVTMPFS — CONFIG_DEVTMPFS + CONFIG_DEVTMPFS_MOUNT

CONFIG_DEVTMPFS — faz o kernel criar automaticamente um filesystem temporário com os device nodes em /dev. Sem isso você teria que criar /dev/sda, /dev/tty, etc. manualmente.

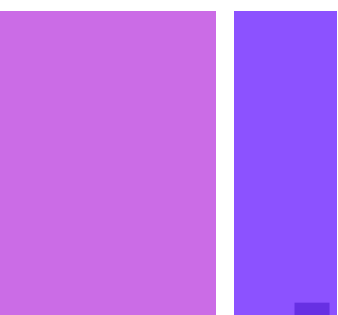
CONFIG_DEVTMPFS_MOUNT — faz o kernel montar esse filesystem em /dev automaticamente durante o boot, antes de qualquer processo do userspace rodar.

Sem os dois juntos:

kernel cria os device nodes

- mas não monta /dev automaticamente
- /dev está vazio no boot
- nenhum programa consegue acessar hardware
- shell abre mas ls /dev = vazio

São dois configs separados que precisam dos dois y.



INITRAMFS — CONFIG_BLK_DEV_INITRD

Permite o kernel carregar e usar um initramfs — um filesystem mínimo embutido na RAM durante o boot, antes de montar o rootfs real.

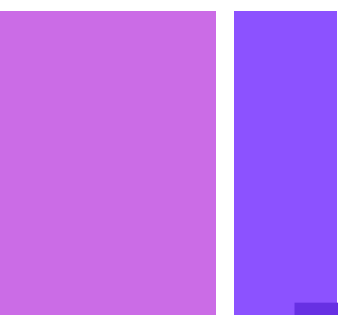
Sem isso:

GRUB carrega bzImage + initramfs

- kernel ignora o initramfs
- tenta montar rootfs diretamente
- se não conseguir → kernel panic imediato

No QEMU (e na maioria das distros modernas) o boot passa pelo initramfs. Sem isso o kernel não sabe como usá-lo.





ELF — CONFIG_BINFMT_ELF

O kernel precisa saber como carregar e executar um binário ELF na memória. Sem isso ele não sabe o que fazer quando alguém chama `execve()`.

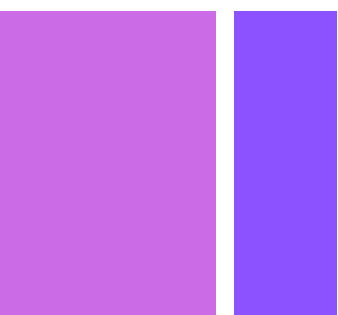
Sem isso:

shell chama `/bin/ls`

- kernel lê o arquivo
- não reconhece o formato ELF
- "Exec format error"
- nenhum binário roda — nem `sh`, nem `ls`, nem nada

É o mais fundamental de todos — sem ele o sistema não consegue executar literalmente nenhum programa.





SCRIPTS / SHEBANGS — CONFIG_BINFMT_SCRIPT

Permite executar arquivos que começam com `#!/caminho/para/interpretador`.

Sem isso:

```
#!/bin/shecho "hello"
```

- kernel não reconhece o shebang
- "Exec format error"
- nenhum script shell, Python, Perl roda diretamente

Todo script de init, script de boot, e script de configuração depende disso.

